

Module 5 – Core PHP

PHP Syntax

THEORY EXERCISE:

- Discuss the structure of a PHP script and how to embed PHP in HTML.

Structure of a PHP Script:

1. Opening and Closing Tags:

A PHP script starts with `<?php` and ends with `?>`.

```
<?php
// PHP code here
?>
```

2. Statements:

PHP code is written as statements ending with a semicolon ;.

```
echo "Hello World!";
```

3. Comments:

- Single line: `// comment` or `# comment`
- Multi-line: `/* comment */`

Embedding PHP in HTML:

PHP is often used inside HTML to make pages dynamic.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>PHP Example</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>
    <?php
      echo "Today is " . date("l");
    ?>
  </p>
</body>
</html>
```

- What are the rules for naming variables in PHP?

- A variable always starts with a \$ sign.

```
$name = "Tanisha";
```

- The first character after \$ must be a letter (a–z, A–Z) or underscore _.
✗ Not allowed: \$9num
✓ Allowed: \$num9, \$_value
- Variable names can contain letters, numbers, and underscores.
Example: \$user_name1
- Variable names are case-sensitive.
\$name and \$Name are different.
- No spaces or special characters (@, #, %, -) are allowed.
✗ \$user-name
\$user_name

LAB EXERCISE:

- Write a PHP script to print "Hello, World!" on a web page.

```
<?php
    echo "Hello, World!";
?>
```

3. PHP Variables

THEORY EXERCISE:

- Explain the concept of variables in PHP and their scope.

Variables in PHP

- A variable in PHP is a named storage that holds data which can change during program execution.
- Declared with a \$ sign followed by the variable name.
- Example:

```
$name = "Tanisha";
```

```
$age = 21;
```

- PHP variables are loosely typed → you don't need to declare a data type; it's assigned automatically.

Scope of Variables in PHP

The scope determines where a variable can be accessed in a script.

1. Local Scope

- Variables declared inside a function.
- Accessible only within that function.

```
function test() {
    $x = 10; // local
    echo $x;
}
test();
// echo $x; // Error: undefined
```

2. Global Scope

- Variables declared outside functions.
- Cannot be accessed directly inside functions.
- Use global keyword to access inside a function.

```
$y = 20;
function demo() {
    global $y;
    echo $y;
}
demo();
```

3. Static Scope

- Variables inside functions normally get deleted after execution.
- Using static keyword keeps their value between function calls.

```
function counter() {
    static $count = 0;
    $count++;
    echo $count . "<br>";
}
```

```
}  
counter();  
counter();
```

4. Superglobals

- Special built-in variables accessible everywhere in PHP (global scope).
- Examples: \$_GET, \$_POST, \$_SESSION, \$_COOKIE, \$_SERVER, etc.

LAB EXERCISE:

□ Create a PHP script to declare and initialize different types of variables (integer, float, string, boolean). Display them using echo

```
<?php
```

```
$age = 21;
```

```
$price = 199.99;
```

```
$name = "Tanisha";
```

```
$isStudent = true;
```

```
echo "Integer value: " . $age . "<br>";
```

```
echo "Float value: " . $price . "<br>";
```

```
echo "String value: " . $name . "<br>";
```

```
echo "Boolean value: " . $isStudent . "<br>"; // true = 1, false = empty
```

```
?>
```

4. Super Global Variables

THEORY EXERCISE:

□ What are super global variables in PHP? List at least five super global arrays and their use.

Super Global Variables in PHP

- Superglobals are built-in variables in PHP that are always accessible, regardless of scope (inside or outside functions).
- They are associative arrays and hold information like form data, session data, server details, etc.
- No need to declare them as global.

Five Common Superglobal Arrays & Their Uses

1. \$_GET

- Collects data sent via URL parameters (query string).
- Example: example.php?name=Tanisha

```
echo $_GET['name']; // Outputs: Tanisha
```

2. \$_POST

- Collects data sent via an HTML form using POST method.
- Used for secure data transfer (like login forms).

```
echo $_POST['email'];
```

3. \$_REQUEST

- Contains data from both GET and POST requests, and sometimes cookies.
- Useful when you don't know which method is used.

```
echo $_REQUEST['username'];
```

4. \$_SESSION

- Stores user information across multiple pages during a browsing session.
- Example: saving login state.

```
$_SESSION['user'] = "Tanisha";
```

5. \$_COOKIE

- Stores small pieces of data on the client's browser.
- Used for preferences, login tokens, etc.

```
echo $_COOKIE['theme'];
```

LAB EXERCISE:

□ Create a form that takes a user's name and email. Use the \$_POST super global to display the entered data.

```
<!DOCTYPE html>
<html>
<head>
  <title>User Form</title>
</head>
<body>

  <!-- HTML Form -->
  <form method="post" action="">
    <label>Name:</label>
    <input type="text" name="name" required><br><br>

    <label>Email:</label>
    <input type="email" name="email" required><br><br>

    <input type="submit" value="Submit">
  </form>

  <hr>

  <?php
  // PHP Script to display form data
  if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $name = $_POST['name'];
    $email = $_POST['email'];

    echo "<h3>Entered Data:</h3>";
    echo "Name: " . $name . "<br>";
    echo "Email: " . $email . "<br>";
  }
  ?>
</body>
</html>
```

5. Practical Example : Multiple Tables and SQL Queries LAB EXERCISE:

Create multiple tables and perform queries using:

o SELECT, UPDATE, DELETE, INSERT o WHERE, LIKE, GROUP BY, HAVING o LIMIT, OFFSET, Subqueries, AND, OR, NOT, IN

-- Students table

```
CREATE TABLE Students (
  student_id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50),
```

```
age INT,  
class VARCHAR(10),  
email VARCHAR(100)  
);
```

-- Courses table

```
CREATE TABLE Courses (  
course_id INT PRIMARY KEY AUTO_INCREMENT,  
course_name VARCHAR(50),  
teacher VARCHAR(50)  
);
```

-- Enrollments table (many-to-many relation)

```
CREATE TABLE Enrollments (  
enroll_id INT PRIMARY KEY AUTO_INCREMENT,  
student_id INT,  
course_id INT,  
grade VARCHAR(5),  
FOREIGN KEY (student_id) REFERENCES Students(student_id),  
FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
);
```

INSERT Queries

sql

Copy code

-- Insert students

```
INSERT INTO Students (name, age, class, email) VALUES  
( 'Tanisha', 21, 'BCA', 'tanisha@example.com'),  
( 'Rohit', 22, 'BBA', 'rohit@example.com'),  
( 'Aditi', 20, 'BCA', 'aditi@example.com');
```

-- Insert courses

```
INSERT INTO Courses (course_name, teacher) VALUES  
( 'Database Systems', 'Dr. Sharma'),  
( 'Web Development', 'Ms. Kapoor');
```

-- Insert enrollments

```
INSERT INTO Enrollments (student_id, course_id, grade) VALUES  
(1, 1, 'A'),  
(1, 2, 'B'),  
(2, 1, 'A'),  
(3, 2, 'C');
```

SELECT Queries

sql

Copy code

-- Select all students

```
SELECT * FROM Students;
```

-- Select specific columns

```
SELECT name, email FROM Students;
```

-- Select with WHERE

```
SELECT * FROM Students WHERE age > 20;
```

-- Using LIKE (pattern matching)

SELECT * FROM Students WHERE name LIKE 'A%'; -- names starting with A
UPDATE and DELETE

sql

Copy code

-- Update email

UPDATE Students SET email = 'tanisha123@example.com' WHERE student_id = 1;

-- Delete a student

DELETE FROM Students WHERE student_id = 3;
GROUP BY and HAVING

sql

Copy code

-- Count students in each class

SELECT class, COUNT(*) AS total_students
FROM Students
GROUP BY class;

-- Having example (classes with more than 1 student)

SELECT class, COUNT(*) AS total_students
FROM Students
GROUP BY class
HAVING COUNT(*) > 1;
LIMIT and OFFSET

sql

Copy code

-- Show first 2 students

SELECT * FROM Students LIMIT 2;

-- Show next 2 students (pagination)

SELECT * FROM Students LIMIT 2 OFFSET 2;
Subquery

sql

Copy code

-- Find students enrolled in 'Database Systems'

SELECT name
FROM Students
WHERE student_id IN (
 SELECT student_id
 FROM Enrollments
 WHERE course_id = (
 SELECT course_id FROM Courses WHERE course_name = 'Database Systems'
)
);

AND, OR, NOT, IN

sql

Copy code

-- AND condition

SELECT * FROM Students WHERE age > 20 AND class = 'BCA';

-- OR condition

```
SELECT * FROM Students WHERE class = 'BCA' OR class = 'BBA';
```

-- NOT condition

```
SELECT * FROM Students WHERE NOT class = 'BBA';
```

-- IN condition

```
SELECT * FROM Students WHERE student_id IN (1, 3);
```

6. Conditions, Events, and Flows

THEORY EXERCISE:

- Explain how conditional statements work in PHP.

Conditional statements are used to make decisions in a program.

- They execute different blocks of code based on conditions (true/false).
- Conditions are usually comparisons (==, !=, >, <, >=, <=) or logical checks (&&, ||, !).

7. If Condition and If-Else

If LAB EXERCISE:

- Write a PHP program to determine if a number is even or odd using if conditions.

```
<?php
// Example number
$num = 7;

if ($num % 2 == 0) {
    echo $num . " is Even";
} else {
    echo $num . " is Odd";
}
?>
```

8. Practical Example:

Calculator and Day Finder

LAB EXERCISE:

```
<!DOCTYPE html>
<html>
<head>
    <title>Simple Calculator</title>
</head>
<body>
    <h2>PHP Simple Calculator</h2>

    <!-- Calculator Form -->
    <form method="post" action="">
        <input type="number" name="num1" placeholder="Enter first number" required>
        <input type="number" name="num2" placeholder="Enter second number" required>

        <select name="operator" required>
            <option value="+">Addition (+)</option>
            <option value="-">Subtraction (-)</option>
            <option value="*">Multiplication (*)</option>
            <option value="/">Division (/)</option>
        </select>
```

```

    <input type="submit" name="calculate" value="Calculate">
</form>

<hr>

<?php
if (isset($_POST['calculate'])) {
    $num1 = $_POST['num1'];
    $num2 = $_POST['num2'];
    $op = $_POST['operator'];
    $result = 0;

    if ($op == "+") {
        $result = $num1 + $num2;
        echo "Result: $num1 + $num2 = $result";
    } elseif ($op == "-") {
        $result = $num1 - $num2;
        echo "Result: $num1 - $num2 = $result";
    } elseif ($op == "*") {
        $result = $num1 * $num2;
        echo "Result: $num1 × $num2 = $result";
    } elseif ($op == "/") {
        if ($num2 != 0) {
            $result = $num1 / $num2;
            echo "Result: $num1 ÷ $num2 = $result";
        } else {
            echo "Error: Division by zero is not allowed.";
        }
    } else {
        echo "Invalid operator selected.";
    }
}
?>
</body>
</html>

```

Day Finder: Write a script that finds the current day. If it is Sunday, print "Happy Sunday."

```

<?php

$day = date("l"); // Returns full day name (e.g., Sunday, Monday)

if ($day == "Sunday") {
    echo "Happy Sunday!";
} else {
    echo "Today is " . $day;
}
?>

```

9. Switch Case and Ternary Operator

LAB EXERCISE:

1. Restaurant Food Category Program: Use a switch case to display the category

(Starter/Main Course/Dessert) and dish based on user selection.

```
<!DOCTYPE html>
<html>
<head>
  <title>Restaurant Food Category</title>
</head>
<body>
  <h2>Restaurant Menu</h2>

  <!-- User Selection Form -->
  <form method="post" action="">
    <label>Select a Dish:</label>
    <select name="dish" required>
      <option value="soup">Tomato Soup</option>
      <option value="pasta">Pasta Alfredo</option>
      <option value="icecream">Chocolate Ice Cream</option>
    </select>
    <input type="submit" name="submit" value="Show Category">
  </form>

  <hr>

  <?php
  if (isset($_POST['submit'])) {
    $dish = $_POST['dish'];

    switch ($dish) {
      case "soup":
        echo "Category: Starter <br> Dish: Tomato Soup";
        break;

      case "pasta":
        echo "Category: Main Course <br> Dish: Pasta Alfredo";
        break;

      case "icecream":
        echo "Category: Dessert <br> Dish: Chocolate Ice Cream";
        break;

      default:
        echo "Invalid selection.";
    }
  }
  ?>
</body>
</html>
```

2.Ternary Operator Example: Write a script using the ternary operator to display a message if the age is greater than 18.

```
<?php
$age = 20; // Example age
```

```
$message = ($age > 18) ? "You are above 18." : "You are 18 or younger.";
```

```
echo $message;
```

```
?>
```

3.Color Selector: Write a program to display the name of a color based on user input (red, green, blue).

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Color Selector</title>
```

```
</head>
```

```
<body>
```

```
    <h2>Select a Color</h2>
```

```
    <!-- Form for user input -->
```

```
    <form method="post" action="">
```

```
        <label>Enter a color (red, green, blue): </label>
```

```
        <input type="text" name="color" required>
```

```
        <input type="submit" name="submit" value="Show Color">
```

```
    </form>
```

```
    <hr>
```

```
    <?php
```

```
    if (isset($_POST['submit'])) {
```

```
        $color = strtolower($_POST['color']); // convert to lowercase for easy matching
```

```
        if ($color == "red") {
```

```
            echo "You selected: Red";
```

```
        } elseif ($color == "green") {
```

```
            echo "You selected: Green";
```

```
        } elseif ($color == "blue") {
```

```
            echo "You selected: Blue";
```

```
        } else {
```

```
            echo "Invalid color. Please enter red, green, or blue.";
```

```
        }
```

```
    }
```

```
    ?>
```

```
</body>
```

```
</html><!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title>Color Selector</title>
```

```
</head>
```

```
<body>
```

```
    <h2>Select a Color</h2>
```

```
    <!-- Form for user input -->
```

```
    <form method="post" action="">
```

```
        <label>Enter a color (red, green, blue): </label>
```

```
        <input type="text" name="color" required>
```

```
        <input type="submit" name="submit" value="Show Color">
```

```

</form>

<hr>

<?php
if (isset($_POST['submit'])) {
    $color = strtolower($_POST['color']); // convert to lowercase for easy matching

    if ($color == "red") {
        echo "You selected: Red";
    } elseif ($color == "green") {
        echo "You selected: Green";
    } elseif ($color == "blue") {
        echo "You selected: Blue";
    } else {
        echo "Invalid color. Please enter red, green, or blue.";
    }
}
?>
</body>
</html>

```

10. Loops: Do-While, For Each, For Loop

THEORY EXERCISE:

□ Discuss the difference between for loop, foreach loop, and do-while loop in PHP.

LAB EXERCISE:

1. For Loop: Write a script that displays numbers from 1 to 10 on a single line.

```

<?php
for ($i = 1; $i <= 10; $i++) {
    echo $i . " ";
}
?>

```

2. For Loop (Addition): Add all integers from 0 to 30 and display the total.

```

<?php
$sum = 0;

for ($i = 0; $i <= 30; $i++) {
    $sum += $i; // add each number to sum
}

echo "The total sum of integers from 0 to 30 is: " . $sum;
?>

```

3. Chessboard Pattern: Use a nested loop to create a chessboard pattern (8x8 grid).

```

<!DOCTYPE html>
<html>
<head>
    <title>Chessboard Pattern</title>
    <style>
        table {
            border-collapse: collapse;

```

```

    }
    td {
        width: 50px;
        height: 50px;
    }
    .black {
        background-color: black;
    }
    .white {
        background-color: white;
    }
</style>
</head>
<body>
    <h2>8x8 Chessboard Pattern</h2>
    <table border="1">
        <?php
            for ($row = 1; $row <= 8; $row++) {
                echo "<tr>";
                for ($col = 1; $col <= 8; $col++) {

                    if (($row + $col) % 2 == 0) {
                        echo "<td class='white'></td>";
                    } else {
                        echo "<td class='black'></td>";
                    }
                }
                echo "</tr>";
            }
        ?>
    </table>
</body>
</html>

```

Various Patterns: Generate different patterns using loops.

```

<?php
for ($i = 1; $i <= 5; $i++) {
    for ($j = 1; $j <= $i; $j++) {
        echo "* ";
    }
    echo "<br>";
}
?>

```

Output:

markdown

Copy code

```

*
* *
* * *
* * * *
* * * * *

```



2. Inverted Triangle

php

Copy code

<?php

```

for ($i = 5; $i >= 1; $i--) {
    for ($j = 1; $j <= $i; $j++) {
        echo "* ";
    }
    echo "<br>";
}
?>

```

Output:

markdown

Copy code

```

* * * * *
* * * *
* * *
* *
*

```



3. Pyramid Pattern

php

Copy code

<?php

```

for ($i = 1; $i <= 5; $i++) {
    // Print spaces
    for ($s = 1; $s <= 5 - $i; $s++) {
        echo "&nbsp;&nbsp;&nbsp;";
    }

    for ($j = 1; $j <= (2 * $i - 1); $j++) {
        echo "*";
    }
    echo "<br>";
}
?>

```

Output:

markdown

Copy code

```

*
***
*****
*****
*****

```

12. PHP Date-Time Function

LAB EXERCISE:

- ☐ Write a script to display the current date and time in different formats.

```

<!DOCTYPE html>
<html>
<head>
    <title>Date and Time Formats</title>
</head>
<body>
    <h2>Current Date and Time in Different Formats</h2>

    <?php

        $now = date("Y-m-d H:i:s");

        echo "Default format (Y-m-d H:i:s): " . $now . "<br><br>";
        echo "1. Date in d/m/Y format: " . date("d/m/Y") . "<br>";
        echo "2. Date in m-d-Y format: " . date("m-d-Y") . "<br>";
        echo "3. Full Day, Date & Month: " . date("l, d F Y") . "<br>";
        echo "4. Short Day, Date & Month: " . date("D, d M Y") . "<br><br>";
        echo "5. Time in 24-hour format: " . date("H:i:s") . "<br>";
        echo "6. Time in 12-hour format: " . date("h:i:s A") . "<br>";
        echo "7. Hour and Minutes only: " . date("H:i") . "<br><br>";
        echo "8. Full Date and Time: " . date("l, d F Y - h:i:s A") . "<br>";
    ?>
</body>
</html>

```

13. Header Function

THEORY EXERCISE: ☐ What is the header function in PHP and how is it used?

The header() function in PHP is used to send raw HTTP headers to the browser before any output is sent.

- It allows you to control HTTP responses, such as redirection, content type, caching, etc.
- Syntax:

```
header(string $header, bool $replace = true, int $response_code = 0)
```

LAB EXERCISE:

☐ Redirect users to another page using the header() function.

redirect.php

```

<?php
header("Location: welcome.php");
exit;
?>

```

welcome.php

```

<!DOCTYPE html>
<html>
<head>
    <title>Welcome Page</title>
</head>
<body>
    <h2>You have been redirected successfully!</h2>
</body>
</html>

```

14. Include and Require

☐ Explain the difference between include and require in PHP.
include:

- If the file is not found, PHP gives a warning but the script will continue to run.

require:

- If the file is not found, PHP gives a fatal error and stops script execution.

LAB EXERCISE:

- Use include and require to insert common header and footer files into multiple PHP pages.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>My Website</title>
```

```
</head>
```

```
<body>
```

```
  <header style="background: lightblue; padding: 10px; text-align: center;">
```

```
    <h2>Welcome to My Website</h2>
```

```
  </header>
```

```
  <hr>
```

Step 2: Create footer.php

php

```
<hr>
```

```
<footer style="background: lightgray; padding: 10px; text-align: center;">
```

```
  <p>&copy; 2025 My Website. All rights reserved.</p>
```

```
</footer>
```

```
</body>
```

```
</html>
```

Step 3: Create home.php (using include)

php

```
<?php include("header.php"); ?>
```

```
<h3>Home Page</h3>
```

```
<p>This is the home page content.</p>
```

```
<?php include("footer.php"); ?>
```

Step 4: Create about.php (using require)

php

```
<?php require("header.php"); ?>
```

```
<h3>About Page</h3>
```

```
<p>This is the about page content.</p>
```

```
<?php require("footer.php"); ?>
```

15. Practical Example: Calculator, Factorial, String Reverse

1. Calculator: Create a calculator using user-defined functions.

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
  <title>PHP Calculator</title>
```

```
</head>
```

```
<body>
```

```
  <h2>Simple Calculator</h2>
```

```
<form method="post">
  <input type="number" name="num1" placeholder="Enter first number" required>
  <input type="number" name="num2" placeholder="Enter second number" required>
  <select name="operator">
    <option value="add">+</option>
    <option value="sub">-</option>
    <option value="mul">*</option>
    <option value="div">/</option>
  </select>
  <input type="submit" name="calculate" value="Calculate">
</form>
```

```
<?php
```

```
function add($a, $b) {
    return $a + $b;
}
```

```
function subtract($a, $b) {
    return $a - $b;
}
```

```
function multiply($a, $b) {
    return $a * $b;
}
```

```
function divide($a, $b) {
    if ($b == 0) {
        return "Error: Division by zero!";
    }
    return $a / $b;
}
```

```
if (isset($_POST['calculate'])) {
    $num1 = $_POST['num1'];
    $num2 = $_POST['num2'];
    $operator = $_POST['operator'];
```

```
    switch ($operator) {
        case "add":
            echo "<h3>Result: " . add($num1, $num2) . "</h3>";
            break;
        case "sub":
            echo "<h3>Result: " . subtract($num1, $num2) . "</h3>";
            break;
        case "mul":
            echo "<h3>Result: " . multiply($num1, $num2) . "</h3>";
            break;
        case "div":
            echo "<h3>Result: " . divide($num1, $num2) . "</h3>";
            break;
    }
```



```

        default:
            echo "<h3>Invalid Operator</h3>";
    }
}
?>
</body>
</html>

```

2. Factorial: Write a function that finds the factorial of a number using recursion.

```

<!DOCTYPE html>
<html>
<head>
    <title>Factorial Using Recursion</title>
</head>
<body>
    <h2>Factorial Calculator</h2>
    <form method="post">
        <input type="number" name="num" placeholder="Enter a number" required>
        <input type="submit" name="find" value="Find Factorial">
    </form>

    <?php

    function factorial($n) {
        if ($n <= 1) {
            return 1;
        } else {
            return $n * factorial($n - 1);
        }
    }

    if (isset($_POST['find'])) {
        $num = $_POST['num'];
        echo "<h3>Factorial of $num is: " . factorial($num) . "</h3>";
    }
    ?>
</body>
</html>

```

3.String Reverse: Reverse a string without using built-in functions.

```

<!DOCTYPE html>
<html>
<head>
    <title>String Reverse</title>
</head>
<body>
    <h2>Reverse a String (Without Built-in Functions)</h2>
    <form method="post">
        <input type="text" name="str" placeholder="Enter a string" required>
        <input type="submit" name="reverse" value="Reverse">
    </form>

    <?php

```

```

function reverseString($string) {
    $reversed = "";
    $length = 0;

    while (isset($string[$length])) {
        $length++;
    }

    for ($i = $length - 1; $i >= 0; $i--) {
        $reversed .= $string[$i];
    }

    return $reversed;
}

if (isset($_POST['reverse'])) {
    $str = $_POST['str'];
    echo "<h3>Original String: $str</h3>";
    echo "<h3>Reversed String: " . reverseString($str) . "</h3>";
}
?>
</body>
</html>

```

4. Download File: Create a button that allows users to download a file.

```

<!DOCTYPE html>
<html>
<head>
    <title>File Download</title>
</head>
<body>
    <h2>Download File Example</h2>

    <!-- Download Button -->
    <form method="post">
        <button type="submit" name="download">Download Sample File</button>
    </form>

    <?php
    if (isset($_POST['download'])) {
        $file = "sample.txt";
        if (file_exists($file)) {
            header("Content-Description: File Transfer");
            header("Content-Type: application/octet-stream");
            header("Content-Disposition: attachment; filename=\"" . basename($file) . "\"");
            header("Expires: 0");
            header("Cache-Control: must-revalidate");
            header("Pragma: public");
            header("Content-Length: " . filesize($file));

```

```

        flush();
        readfile($file);
        exit;
    } else {
        echo "<p style='color:red;'>File not found!</p>";
    }
}
?>
</body>
</html>

```

16. PHP Expressions, Operations, and String Functions

THEORY EXERCISE:

□ Explain what PHP expressions are and give examples of arithmetic and logical operations. An expression in PHP is anything that produces a value. It can be a simple number, a variable, a function call, or a combination of values, operators, and functions.

Example:

```
$x = 5 + 10;
```

```
$y = $x * 2;
```

Types of PHP Expressions

1. Arithmetic Expressions → Perform mathematical operations
2. Logical Expressions → Return true/false based on conditions

Arithmetic Expressions

```
$a = 10;
```

```
$b = 3;
```

```
echo $a + $b;
```

```
echo $a - $b;
```

```
echo $a * $b;
```

```
echo $a / $b;
```

```
echo $a % $b;
```

```
echo $a ** $b;
```

? Logical Expressions

Logical expressions are used in conditions (if, while, etc.), they return true (1) or false (0).

```
$x = 10;
```

```
$y = 20;
```

```
var_dump($x == $y);
```

```
var_dump($x != $y);
```

```
var_dump($x < $y);
```

```
var_dump($x > $y);
```

```
var_dump($x <= $y);
```

```
var_dump($x >= $y);
```

```
var_dump($x < 15 && $y > 15);
```

```
var_dump($x < 5 || $y > 15);
```

```
var_dump(!($x == 10));
```

LAB EXERCISE:

□ Write a script to perform various string operations like concatenation, substring extraction, and string length determination.

```
<?php
```

```
$str1 = "Hello";  
$str2 = "World";  
$concat = $str1 . " " . $str2;  
$substring = substr($concat, 0, 5);  
$length = strlen($concat);  
echo "<h3>String Operations</h3>";  
echo "First String: $str1 <br>";  
echo "Second String: $str2 <br>";  
echo "Concatenation: $concat <br>";  
echo "Substring (first 5 chars): $substring <br>";  
echo "Length of concatenated string: $length <br>";  
?>
```